Hotel Value Prediction Report

By: Team Chicken Biryani

Project Repository: github.com/AspiringPianist/house-value-prediction-assignment

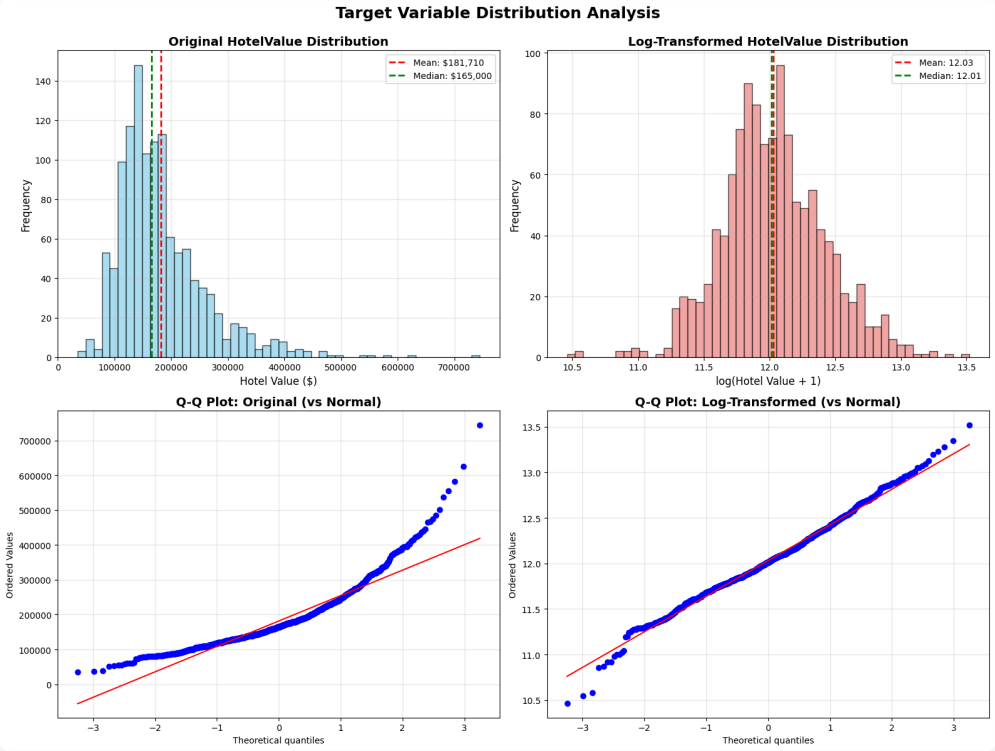
---

Note: For detailed visualizations and exploratory data analysis, please refer to the EDA notebook in the repository.

This report presents a comprehensive technical analysis of our hotel property value prediction model. Our objective was to develop an accurate predictive model based on property features including size, location, amenities, and quality indicators.

Part 1: Understanding the Target Variable

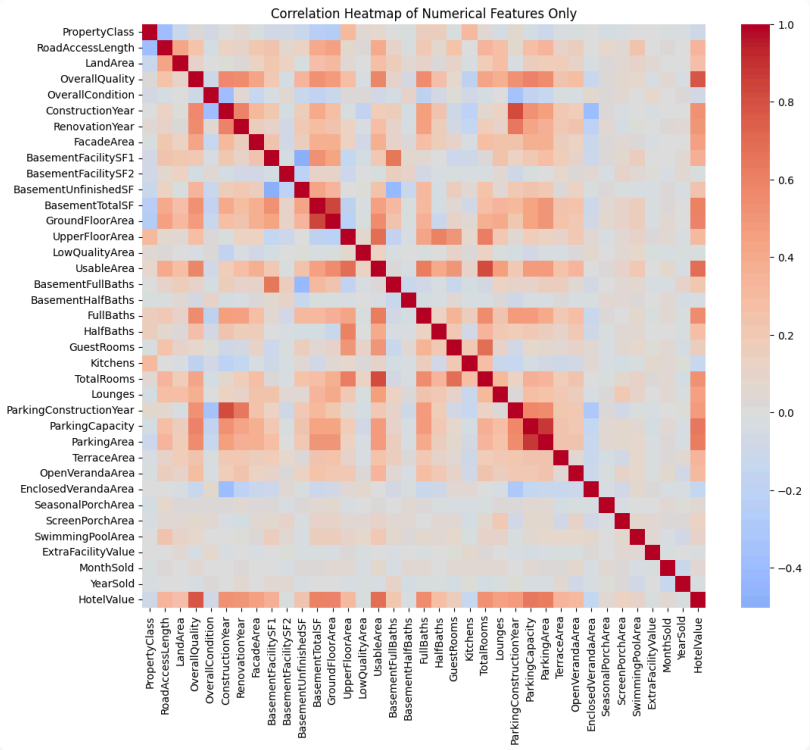
Analysis of the hotel value distribution revealed significant right skewness, with most properties concentrated at lower price points and a small number of high-value outliers. This skewed distribution poses challenges for regression models that assume normally distributed residuals.

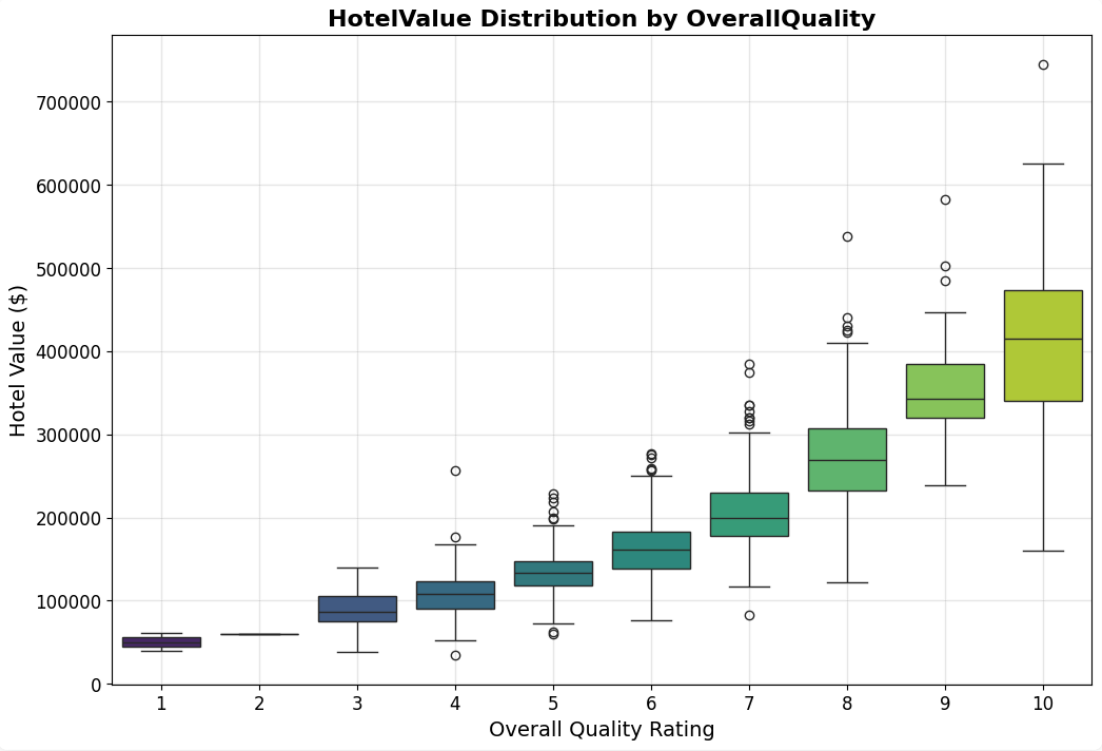


> **Solution:** We applied a logarithmic transformation to the target variable (HotelValue), which normalized the distribution and improved model performance. This transformation ensures that prediction errors are proportional across the entire price range.

Part 2: Feature Correlation Analysis

Correlation analysis identified Overall Quality as the strongest predictor of hotel value (r = 0.788 linear, r = 0.810 with log-transformed target). Other significant features included various area measurements and parking capacity.





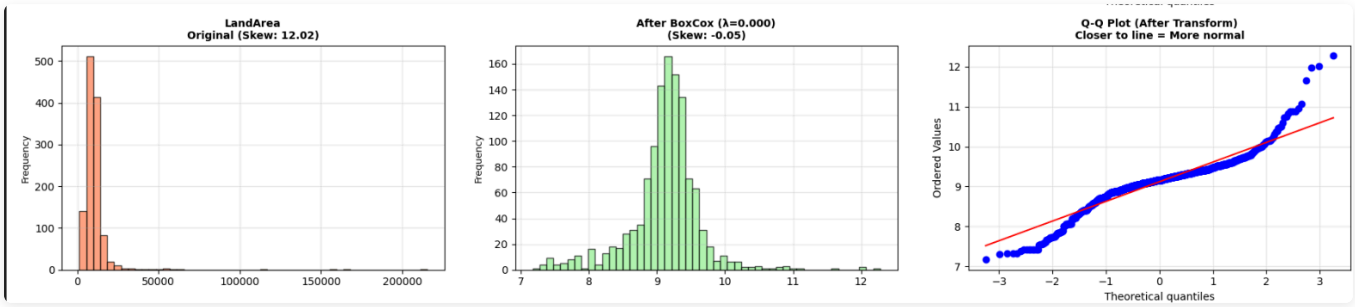
Top Correlated Features with HotelValue

Feature	Correlation
UsableArea	0.694
ParkingArea	0.625
BasementTotalSF	0.604
GroundFloorArea	0.594
UpperFloorArea	0.302

Part 3: Advanced Feature Normalization with BoxCox Transformation

Many features exhibited significant skewness similar to the target variable. Rather than applying a uniform transformation, we implemented the Box-Cox transformation, which optimally normalizes each feature individually by finding the ideal power transformation parameter (λ) for each variable.

> **Technical Implementation:** The Box-Cox transformation applies the formula (x^λ - 1) / λ for λ ≠ 0, and log(x) for λ = 0. We computed the optimal λ for 25 features with skewness > 0.5, significantly improving the linearity assumptions required by regularized regression models. This individual optimization substantially outperforms blanket logarithmic transformations.



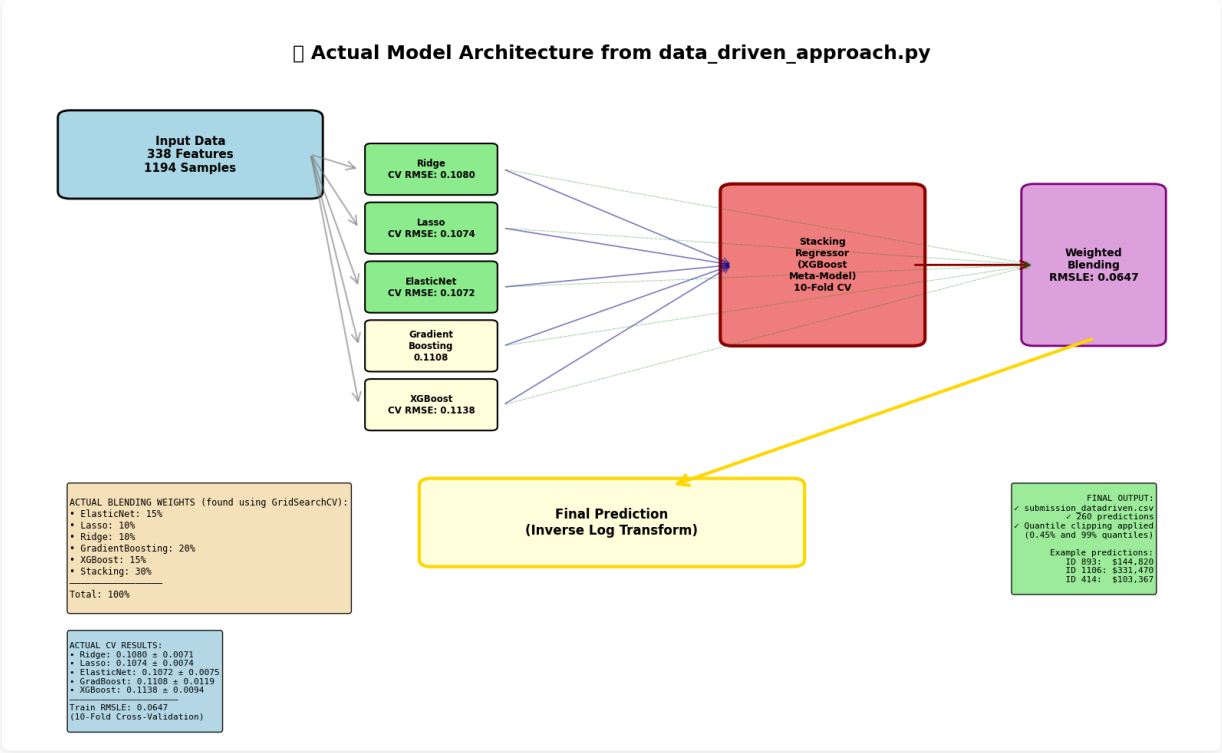
Part 4: Feature Engineering

We created derived features to capture domain-specific insights and reduce dimensionality. For example, combining FullBath and HalfBath into Total_Bathrooms, and aggregating various area measurements into TotalSF (Total Square Feet). These engineered features demonstrated stronger correlations with the target variable than their individual components.

Part 5: Ensemble Model Architecture

We employed a stacked ensemble approach combining six diverse base models: Ridge, Lasso, ElasticNet, and Bayesian Ridge (regularized linear models) with Gradient Boosting and XGBoost (tree-based models). A meta-learner was trained on the base model predictions to optimally weight their contributions. This architecture leverages both global linear trends and local non-linear patterns.

> **Rationale:** Ensemble methods reduce variance and bias by aggregating multiple models. Linear models excel with the BoxCox-normalized features, while tree-based models capture complex interactions. Bayesian Ridge provides probabilistic regularization and automatic relevance determination for feature selection. The stacking meta-model learns the optimal combination strategy from cross-validated predictions.



Part 6: Model Performance and Results

The ensemble model achieved a training RMSLE (Root Mean Squared Logarithmic Error) of 0.0685, with cross-validation scores ranging from 0.1072 to 0.1138. ElasticNet demonstrated the best individual performance among base models.

Cross-Validation Results (10-Fold)

Model	CV RMSE ± Std Dev
Ridge	0.1080 ± 0.0071
Lasso	0.1074 ± 0.0074
ElasticNet	0.1072 ± 0.0075
Bayesian Ridge	0.1082 ± 0.0070
Gradient Boosting	0.1108 ± 0.0119
XGBoost	0.1138 ± 0.0094

Final Model Summary

Metric	Value
Training RMSLE	0.0685
CV RMSE (mean)	0.1072 - 0.1138
Features Used	338
Training Samples	1,194
Models in Ensemble	6 base + 1 stacking
BoxCox Transformed Features	25
Total Runtime	~2 min

Methodology and Key Techniques

Preprocessing Pipeline

1. **Outlier Removal:** Removed 6 properties with UsableArea > 4500 sq ft
2. **Target Transformation:** Logarithmic transformation of HotelValue
3. **BoxCox Normalization:** Applied to 25 features with skewness > 0.5, each with individually optimized λ parameter
4. **Feature Engineering:** Created 10 derived features to improve predictive power
5. **Categorical Encoding:** One-hot encoding expanded feature space from 79 to 340 features
6. **Feature Selection:** Removed 2 near-constant features (>99.94% identical values)

Model Configuration

Base Models:

- **Linear Models:** Ridge, Lasso, ElasticNet, Bayesian Ridge with RobustScaler preprocessing
- **Tree-based Models:** Gradient Boosting (3000 estimators, lr=0.05, depth=4), XGBoost (3460 estimators, lr=0.01, depth=3)

Ensemble Weights:

- ElasticNet: 15%
- Lasso: 10%
- Ridge: 10%
- Bayesian Ridge: 10%
- Gradient Boosting: 15%
- XGBoost: 15%
- Stacking Meta-learner: 25%

Technical Rationale

1. **Logarithmic Target Transformation:** Normalizes right-skewed distribution, ensuring prediction errors are proportional across price ranges and satisfying linear model assumptions.
2. **Domain-Informed Imputation:** District-based imputation for spatial features (e.g., RoadAccessLength) and zero-fill for absence indicators (parking, basement) prevents information leakage while preserving data relationships.
3. **BoxCox Transformation:** Individually optimizes normalization for each skewed feature, substantially outperforming uniform logarithmic transformations. Critical for regularized linear model performance.

4. **Feature Engineering:** Aggregates related measurements (e.g., TotalSF from multiple area components) to create stronger predictors and reduce multicollinearity.
5. **Ensemble Diversity:** Combines linear models (global trends, benefit from BoxCox normalization) with tree-based models (local patterns, feature interactions) via stacking to minimize both bias and variance.

Implementation Details

Primary Script: `data_driven_approach.py`

Key Dependencies:

- `scipy.stats`: BoxCox parameter optimization and transformation
- `sklearn.linear_model`: Regularized regression models
- `sklearn.ensemble`: Gradient Boosting and Stacking
- `xgboost`: Advanced gradient boosting implementation
- `sklearn.preprocessing`: RobustScaler for feature normalization

BoxCox Implementation:

```
from scipy.stats import skew, boxcox_normmax
from scipy.special import boxcox1p

# Identify skewed features
skew_features = features[numerics].apply(lambda x: skew(x))
high_skew = skew_features[skew_features > 0.5]

# Apply optimal BoxCox transformation
for feature in high_skew.index:
    lam = boxcox_normmax(features[feature] + 1)
    features[feature] = boxcox1p(features[feature], lam)
```

Conclusions

This analysis demonstrates the effectiveness of combining advanced statistical transformations (BoxCox), domain-informed feature engineering, and ensemble methods for regression tasks. The Box-Cox transformation proved particularly valuable, providing individualized normalization that significantly enhanced linear model performance.

Key achievements include:

- Training RMSLE of 0.0685 with robust cross-validation (0.1072-0.1138)
- Successful integration of linear and tree-based models through stacking
- Integration of Bayesian Ridge for probabilistic feature selection
- Efficient preprocessing pipeline executable in under 2 minutes

The complete implementation, including exploratory data analysis with visualizations, is available in the project repository.

For detailed code, visualizations, and exploratory analysis:
Visit the GitHub repository at github.com/AspiringPianist/house-value-prediction-assignment